Laboratory 6, Variant 1

Class group 102, group 16

Anatoli Abahumna and Leonardo Meloni

1 Introduction

The goal of this laboratory was to implement tabular Q-Learning for a reinforcement learning problem from the Gymnasium library. The assigned variant was Variant 1, which uses the `CartPole-v1` environment. In this environment, an agent controls a cart and learns to keep a pole balanced for as many time steps as possible.

Q-Learning is a model-free reinforcement learning algorithm. It estimates the long-term value of taking each action in each state. During training, the agent improves a Q-table by interacting with the environment, receiving rewards, and updating the value of the selected action. A main advantage of Q-Learning is that it is simple and does not require a model of the environment. A disadvantage is that it needs a discrete state representation, so continuous observations must be converted to table indices.

2 Environment Description

`CartPole-v1` has two discrete actions: action 0 pushes the cart left, and action 1 pushes the cart right. The observation has four continuous values: cart position, cart velocity, pole angle, and pole angular velocity. The reward is $+1$ for every step that the pole remains balanced. An episode terminates when the cart position is outside ± 2.4 or the pole angle is outside ± 0.2094 radians, and it is truncated after 500 steps.

Because the observation space is continuous, direct tabular Q-Learning is not possible. The implementation clips each observation dimension and maps it to a finite number of bins. The clipping ranges were:

Observation	Minimum	Maximum	Default bins
Cart position	-4.8	4.8	6
Cart velocity	-3.0	3.0	8
Pole angle	-0.418	0.418	12
Pole angular velocity	-3.5	3.5	12

With the default bin setting $(6, 8, 12, 12)$, the Q-table has shape $6 \times 8 \times 12 \times 12 \times 2$, which gives 13,824 Q-values.

3 Implementation

The program is contained in one Python file. It creates the `CartPole-v1` environment, initializes the Q-table with zeros, trains with epsilon-greedy exploration, evaluates the learned table with greedy actions, and saves metrics and plots.

3.1 Discretization

The continuous observation is clipped to fixed ranges and then converted to bin indices:

```
clipped = np.clip(observation, OBSERVATION_LOW, OBSERVATION_HIGH)
ratios = (clipped - OBSERVATION_LOW) / (OBSERVATION_HIGH - OBSERVATION_LOW)
state = np.floor(ratios * np.asarray(bins)).astype(int)
state = np.clip(state, 0, np.asarray(bins) - 1)
```

3.2 Action Choice

During training, the agent uses an epsilon-greedy policy. With probability ϵ, it explores by selecting a random action. Otherwise, it selects one of the best actions from the current Q-table row.

```
if rng.random() < epsilon:
    action = int(rng.integers(q_table.shape[-1]))
else:
    q_values = q_table[state]
    best_actions = np.flatnonzero(q_values == np.max(q_values))
    action = int(rng.choice(best_actions))
```

3.3 Q-Value Update

After each environment step, the Q-table is updated with the standard Q-Learning rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right)$$

```
old_value = q_table[state + (action,)]
next_best = 0.0 if done else float(np.max(q_table[next_state]))
target = reward + gamma * next_best
q_table[state + (action,)] = old_value + alpha * (target - old_value)
```

For each episode, the program logs total reward, episode length, epsilon value, and the 100-episode rolling average reward. The environment is marked as solved if the rolling average reaches at least 475. Training still runs for the configured number of episodes by default, which makes the experiment comparisons fair.

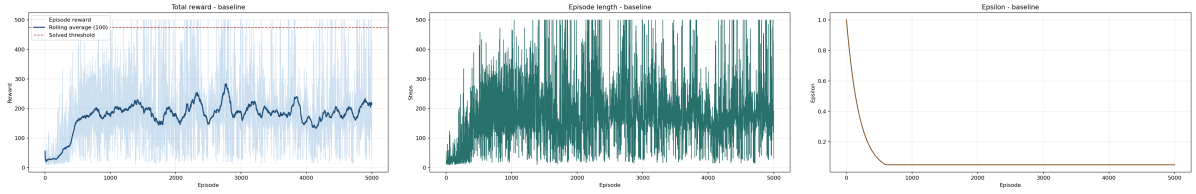


Figure 1: Baseline training metrics: reward, episode length, and epsilon.

4 Experiments and Results

The baseline configuration used 5000 training episodes, learning rate $\alpha = 0.1$, discount factor $\gamma = 0.99$, $\epsilon_{start} = 1.0$, $\epsilon_{min} = 0.05$, $\epsilon_{decay} = 0.995$, bins $(6, 8, 12, 12)$, and seed 42. Each sweep changed one hyperparameter while keeping the others at the baseline values. Evaluation was performed after training for 20 greedy episodes.

4.1 Learning Rate

α	Final rolling	Best rolling	Solved episode	Eval. average
0.05	298.44	318.12	–	398.95
0.10	214.31	283.66	–	197.30
0.20	228.19	320.30	–	314.75

Table 1: Learning rate sweep.

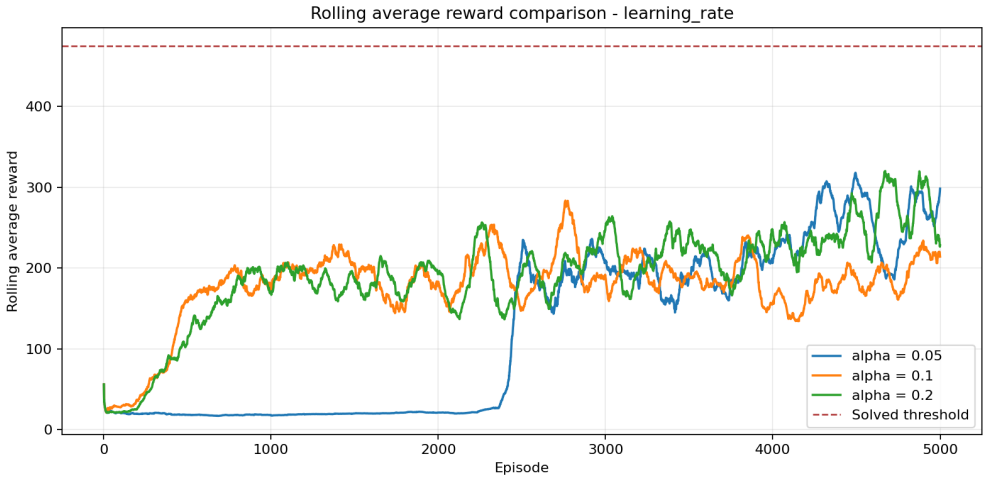


Figure 2: Rolling average reward for different learning rates.

The lower learning rate $\alpha = 0.05$ produced the best final rolling reward and evaluation average. The larger rate $\alpha = 0.2$ achieved a similar best rolling reward, but its final result was less stable. This suggests that smaller updates were better for the coarse discretized state space.

4.2 Discount Factor

γ	Final rolling	Best rolling	Solved episode	Eval. average
0.950	183.80	204.36	–	143.30
0.990	214.31	283.66	–	197.30
0.999	259.33	334.93	–	476.40

Table 2: Discount factor sweep.

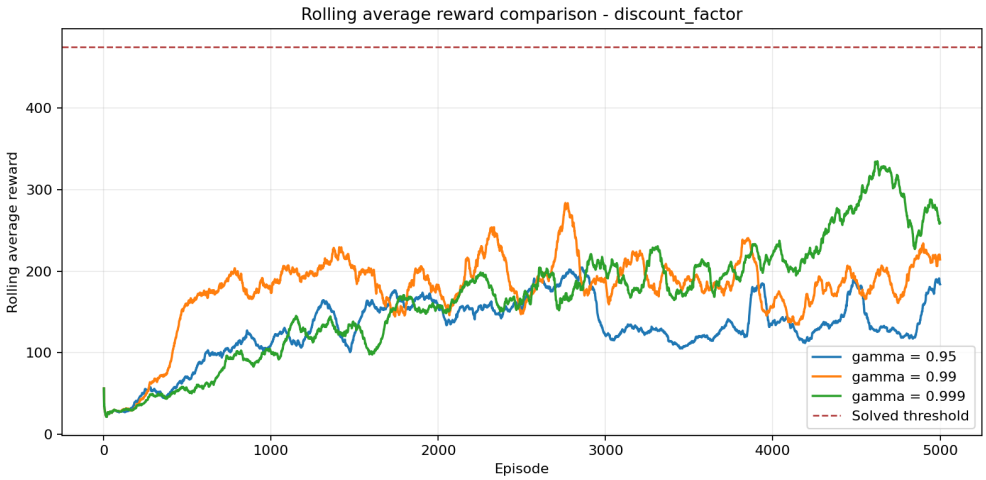


Figure 3: Rolling average reward for different discount factors.

The best evaluation result came from $\gamma = 0.999$. CartPole rewards long balanced episodes, so valuing future rewards strongly helps the agent prefer actions that keep the pole stable for longer. The training rolling average still did not reach 475 because the training policy continued to use at least 5% random actions.

4.3 Initial Exploration Rate

ϵ_{start}	Final rolling	Best rolling	Solved episode	Eval. average
0.5	214.84	227.73	–	230.70
0.8	214.44	280.26	–	210.20
1.0	214.31	283.66	–	197.30

Table 3: Initial exploration rate sweep.

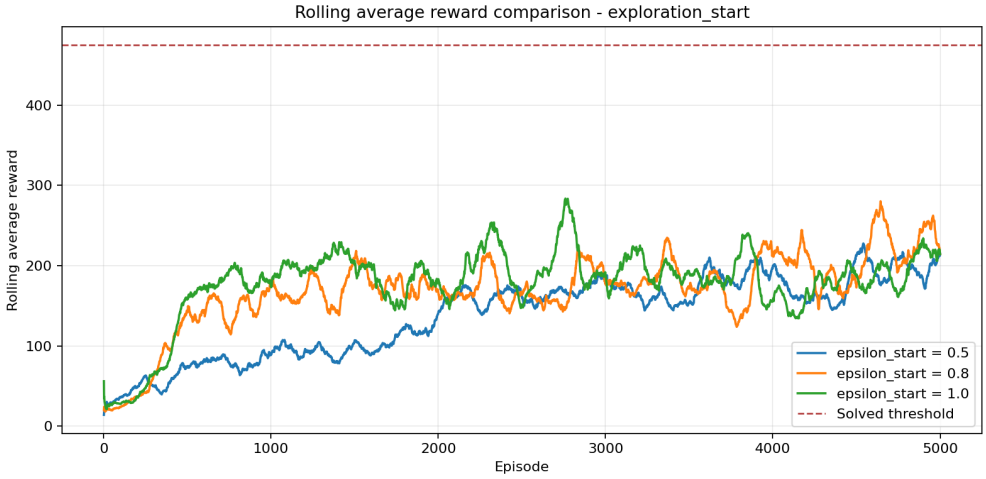


Figure 4: Rolling average reward for different initial epsilon values.

Changing the initial exploration rate had a smaller effect than changing the discount factor or decay rate. Starting at 1.0 explored the most early states, but the final evaluation average was

not the best. Starting at 0.5 produced a slightly better greedy evaluation result, probably because useful actions were exploited earlier.

4.4 Exploration Decay

ϵ_{decay}	Final rolling	Best rolling	Solved episode	Eval. average
0.990	184.87	249.07	–	159.55
0.995	214.31	283.66	–	197.30
0.999	311.67	376.51	–	268.65

Table 4: Exploration decay sweep.

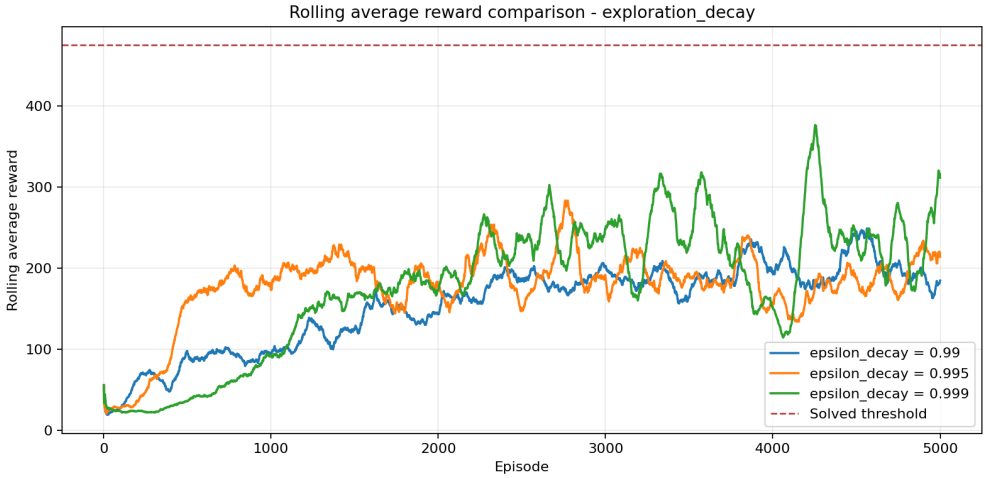


Figure 5: Rolling average reward for different epsilon decay values.

The slower decay value 0.999 gave the best final and best rolling rewards in this sweep. Fast decay reduced exploration too early, so the agent had fewer chances to improve values for less frequently visited states.

4.5 Quantization Size

Bins	Final rolling	Best rolling	Solved episode	Eval. average
$(4, 6, 8, 8)$	228.77	293.29	–	297.95
$(6, 8, 12, 12)$	214.31	283.66	–	197.30
$(8, 10, 14, 14)$	133.25	170.81	–	143.45

Table 5: Quantization size sweep.

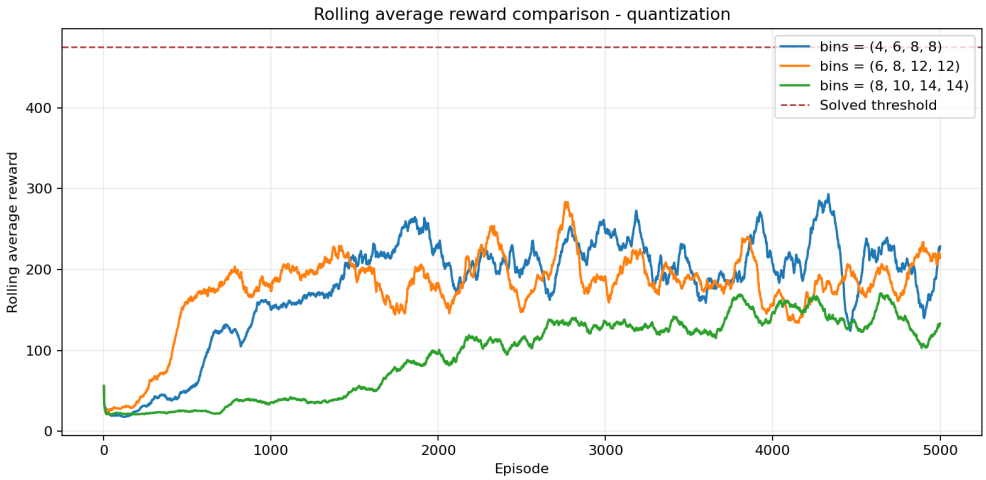


Figure 6: Rolling average reward for different quantization sizes.

The coarser $(4, 6, 8, 8)$ discretization performed better than the default and the finer setting. With a fixed number of episodes, the larger table has more state-action values to estimate, so many bins are visited less often. The coarser table loses detail but learns each state more reliably.

5 Discussion

No run reached the 475 solved threshold based on the 100-episode rolling average during exploratory training. However, the discount factor sweep showed that the learned greedy policy can be much stronger than the exploratory training curve: $\gamma = 0.999$ reached an evaluation average of 476.40 over 20 greedy episodes. This is close to the maximum score of 500 and indicates that the Q-table learned useful balancing behavior.

The most important observations were:

- Smaller learning rate $\alpha = 0.05$ gave the best learning-rate result because updates were smoother.
- Larger discount factor $\gamma = 0.999$ worked best because CartPole requires long-term balance.
- Slower exploration decay $\epsilon_{decay} = 0.999$ improved the rolling reward by allowing more state coverage.
- Coarser quantization learned faster than finer quantization because the Q-table had fewer entries.

The best single changed configuration by greedy evaluation was $\gamma = 0.999$. The best single changed configuration by final training rolling average was $\epsilon_{decay} = 0.999$. These two results show the trade-off between final greedy policy quality and the noisy reward measured during training with remaining random exploration.

6 Conclusion

This laboratory showed how tabular Q-Learning can be applied to a continuous-control environment by discretizing the observations. The main difficulty was choosing bins and exploration parameters that allow enough learning while keeping the table small. The experiments showed that CartPole benefits from valuing future rewards highly and from slower exploration decay. Possible improvements include testing combined best hyperparameters, using more training episodes for

the larger quantization table, and trying non-uniform bins around the most important pole-angle region.